

Interpreting a Bidirectional LSTM-CRF Model for Disease Entity Recognition

Biomedical Named Entity Recognition meets Explainable AI

Authors	Akshat Gupta & Silvia Cunico
Supervision	Prof. Roman Klinger, University of Stuttgart
Domain	Clinical / Biomedical NLP · Sequence Labelling · Interpretability
Primary dataset	NCBI Disease Corpus (parallel gene/protein pipeline)
Code base	Python 3.9+ · TensorFlow/Keras · HuggingFace · seqeval · eli5

ABSTRACT

This report documents a research code base that trains and compares models for recognising disease (and gene/protein) mentions in biomedical text, and then explains their predictions. It spans a from-scratch Naive Bayes baseline, a BiLSTM and a custom linear-chain BiLSTM-CRF, and a modern BioBERT / PubMedBERT transformer baseline, all evaluated with strict entity-level seqeval metrics. On top of recognition, it provides a suite of interpretability tools - LIME, Integrated Gradients, attention and a cross-method saliency-disagreement study - aimed at understanding what sequence models capture that count-based methods cannot.

Contents

1	Executive Summary	5
2	Introduction and Motivation	6
	The recognition task	6
	Why interpretability	6
3	System Architecture Overview	7
	The two original pipelines	7
	Repository layout	7
	Neural pipeline data flow	7
4	Datasets and Data Format	9
	Corpora	9
	File format and a critical quirk	9
	Corpus statistics	9
5	Models and Methodology	10
	Naive Bayes baseline (from scratch)	10
	BiLSTM	10
	BiLSTM-CRF	10
	BioBERT / PubMedBERT	11
	Encoding and preprocessing details	11
6	Evaluation Methodology	12
7	Interpretability	13
	LIME on the BiLSTM	13
	GALE - global aggregation	13
	BioBERT embeddings and t-SNE	13
	Cross-method comparison	13
	Integrated Gradients	13
8	The Saliency-Disagreement Study	14
	Supporting modules	14
	Reference numbers (smoke test)	14
9	The 2026 Refactor: What Changed and Why	15
10	Results	16
	Re-run targets	16
	Historical training curves (legacy BiLSTM)	16
11	How to Run and Reproduce	17
12	Environment and Dependencies	18
13	Limitations and Future Work	19
	Known limitations	19

Future work	19
14 Conclusion	20

1 Executive Summary

Named Entity Recognition (NER) over biomedical text is the task of locating and labelling spans that refer to entities of interest - here, diseases (and, in a parallel pipeline, genes/proteins). This project pursues two coupled goals: (1) build models that tag each token of a sentence with a BIO label, and (2) interpret those models so that their decisions can be inspected and trusted, which is essential in clinical and biomedical settings.

The code base is organised as two historical pipelines plus a 2026 refactor that modernised both evaluation and modelling. The original work (2022) shipped a from-scratch Multinomial Naive Bayes tagger and a Keras BiLSTM, explained with LIME and visualised with BioBERT embeddings under t-SNE. The refactor added a real linear-chain CRF layer, a transformer (BioBERT/PubMedBERT) baseline, strict entity-level evaluation via sequeval, multi-seed reporting, and a cross-method interpretability / saliency-disagreement study.

Key takeaways

- Four model families are supported: Naive Bayes, BiLSTM, BiLSTM-CRF, and BioBERT/PubMedBERT - spanning ~30 years of NER methodology.
- Evaluation was corrected from an inflated token-level accuracy (~0.98, dominated by the majority 'O' class) to strict entity-level sequeval F1, under which an all-'O' predictor correctly scores $F1 = 0$.
- Interpretability is treated as a first-class deliverable: LIME, Integrated Gradients, attention, and Naive-Bayes log-probability weights are compared head-to-head on the same sentences.
- A saliency-disagreement study asks whether explanation methods disagree more on biomedical NER than on general-domain NER, and which linguistic phenomena drive any gap.

2 Introduction and Motivation

2.1 The recognition task

Given a tokenised sentence, the model assigns each token one of three labels in the BIO (also called IOB2) scheme: B-DISEASE marks the first token of a disease mention, I-DISEASE marks a subsequent token inside the same mention, and O marks any token outside an entity. Entity spans are therefore recovered by scanning for a B tag followed by zero or more consecutive I tags of the same type.

Worked example

John	has	been	diagnosed	with	sporadic	T-cell	leukaemia
O	O	O	O	O	B-DISEASE	I-DISEASE	I-DISEASE

Here "sporadic T-cell leukaemia" is a single three-token disease entity. The challenge is that disease mentions are frequently multi-word, contain abbreviations and parenthesised acronyms, and use vocabulary that is far from everyday English - exactly the properties this project later quantifies.

2.2 Why interpretability

A tagger that is accurate but opaque is hard to deploy in biomedical practice. The project therefore explains predictions both locally (why did the model tag this token this way?) and globally (which words, across the corpus, push the model toward a disease label?). The two requested approximation techniques are LIME (local, perturbation-based) and GALE (a global aggregation of local explanations).

Central research question

What do sequence models "see" in NER that a count-based Naive Bayes classifier cannot? The Naive Bayes baseline ignores word order, part-of-speech, and context; the LSTM adds sequential memory and the CRF adds label-transition structure. The interpretability layer is the instrument used to answer this question concretely.

3 System Architecture Overview

The repository contains two parallel, self-contained pipelines plus a set of modern add-ons. Every script is designed to be run from inside the `code/` directory because all paths are relative to it (`../data`, `../models`, `../figures`).

3.1 The two original pipelines

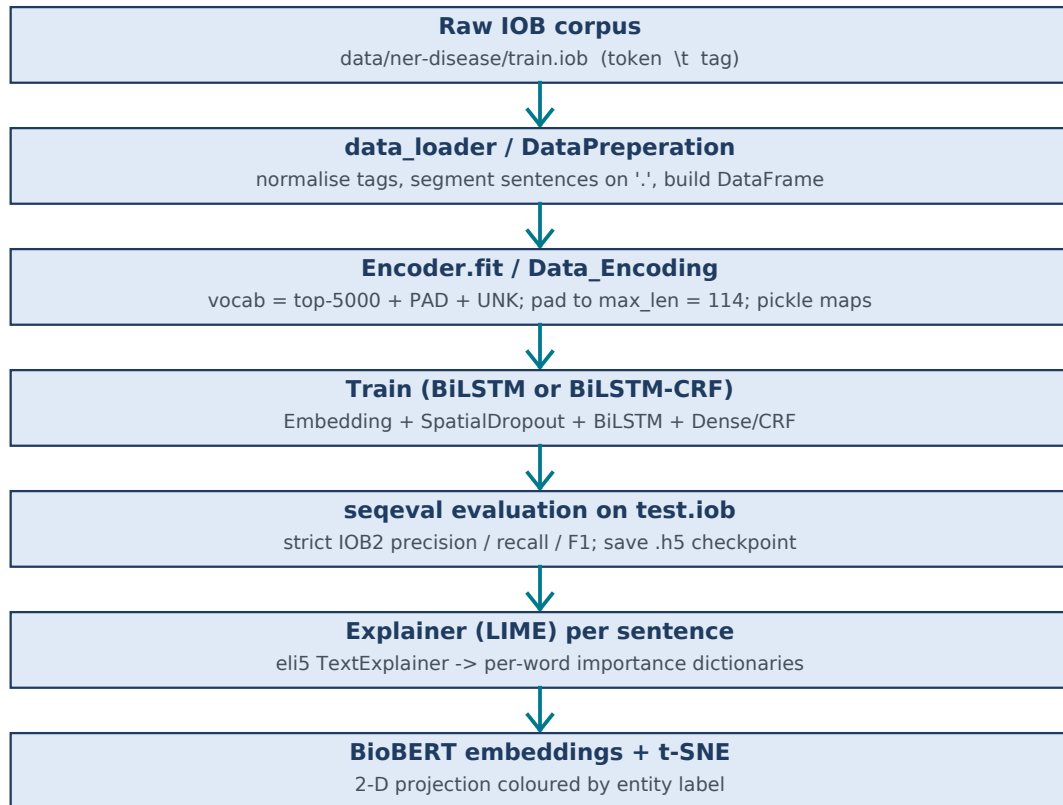
- Naive Bayes baseline - entry point `code/main_NB.py`, model in `code/NaiveBayes.py`. Runs the disease corpus and then the gene/protein corpus back-to-back, scored with custom entity-level metrics.
- Neural pipeline - entry point `code/main_NN.py`. Originally: preprocess raw IOB into a CSV, run exploratory data analysis, train a BiLSTM, generate LIME explanations per word, then compute BioBERT embeddings and t-SNE.

3.2 Repository layout

Path	Role
<code>code/main_NB.py</code>	Naive Bayes pipeline driver (disease + gene)
<code>code/main_NN.py</code>	Neural pipeline CLI (BiLSTM / BiLSTM-CRF, NCBI / BC5CDR)
<code>code/NaiveBayes.py</code>	From-scratch Multinomial Naive Bayes tagger
<code>code/Preprocess.py</code> / <code>DataPreparation.py</code>	IOB readers; CSV + POS feature builder
<code>code/data_loader.py</code>	Unified IOB/CoNLL reader (NCBI, BC5CDR, CoNLL-2003)
<code>code/Train.py</code>	BiLSTM and BiLSTM-CRF construction, training, evaluation
<code>code/crf_layer.py</code>	Self-contained linear-chain CRF Keras layer
<code>code/Evaluation.py</code>	segeval entity-level metrics (+ legacy shims)
<code>code/Explainer.py</code>	LIME explanations over the BiLSTM (via eli5)
<code>code/BioBertEmbeddings.py</code>	BioBERT embeddings + t-SNE visualisation
<code>code/train_biobert.py</code>	BioBERT / PubMedBERT fine-tuning (HuggingFace)
<code>code/multi_seed_runner.py</code>	Train across seeds; report mean +/- std
<code>code/interpretability_compare.py</code>	NB vs LIME vs attention agreement
<code>code/run_disagreement_study.py</code>	Saliency-disagreement experiment driver
<code>code/explainers/</code>	saliency_metrics, phenomena, faithfulness, IG modules
<code>data/</code> , <code>models/</code> , <code>figures/</code>	Corpora, checkpoints (.h5), training plots
<code>archive/</code>	Earlier keras-contrib CRF prototype (not wired in)

3.3 Neural pipeline data flow

`main_NN.py` orchestrates an exact sequence in which each step writes an artifact that the next step consumes:



4 Datasets and Data Format

4.1 Corpora

The disease models are trained on the NCBI Disease corpus; a parallel pipeline reuses the same machinery on a gene/protein corpus (PROTEIN labels). The refactor also adds loaders for BC5CDR (cross-dataset biomedical evaluation) and CoNLL-2003 (the canonical general-domain comparison corpus), neither of which is redistributed in the repository.

4.2 File format and a critical quirk

Input files follow the CoNLL-2003 convention loosely: one token per line, with the token and its tag tab-separated. The NCBI files in this repository use a non-standard tag encoding - tags carry a leading pipe and a trailing newline, e.g. the literal strings '|B-DISEASE\n', '|I-DISEASE\n', '|O\n' (and the PROTEIN equivalents for genes).

Why the quirk matters

The original Naive Bayes and explainer code compares tags against these exact string literals. Any code that strips whitespace or the leading pipe silently breaks label matching. The refactor's `data_loader._clean_tag` normalises every tag to canonical B-DISEASE / I-DISEASE / O form so that seqeval and HuggingFace tokenizers can consume it safely, while the legacy NB path keeps the literal strings it was written against.

4.3 Corpus statistics

Counts below are computed by the repository's own loader (sentences segmented on the '.' token, the same convention used throughout the pipeline). Note that sentence-level segmentation yields counts close to, but not identical to, the published NCBI split (5424 / 923 / 940) because of tokenisation and segmentation differences.

NCBI Disease

Split	Sentences	Tokens	Disease spans (B-)	B-DISEASE	I-DISEASE	O
train	5,915	128,721	5,148	5,148	4,836	118,737
dev	962	22,646	791	791	830	21,025
test	981	23,175	961	961	866	21,348

Gene / Protein

Split	Sentences	Tokens	Protein spans (B-)	B-PROTEIN	I-PROTEIN	O
train	9,707	257,743	9,700	9,700	8,128	239,915
dev	5,565	158,422	8,465	8,465	6,889	143,068
test	5,095	140,006	6,290	6,290	4,801	128,915

The heavy class imbalance is the headline fact: on the NCBI test split, roughly 92% of tokens are 'O'. This is precisely why a token-level accuracy metric is misleading - a degenerate model that never predicts an entity already scores about 0.92 - and why the project moved to entity-level scoring.

5 Models and Methodology

5.1 Naive Bayes baseline (from scratch)

NaiveBayes.py implements a Multinomial Naive Bayes token classifier without external ML libraries. Training counts how often each word occurs under each class (B, I, O), converts counts to add-one (Laplace) smoothed log-probabilities, and stores class log-priors. Prediction sums the relevant log-probabilities per class and returns the arg-max label.

```
# Smoothed class-conditional log-probability (add-one smoothing)
features['B'][word] = log((count + 1) / (count_B + doc_count))

# Prediction: start from log-prior, add evidence, take arg-max
p_B = priori_B + sum(features['B'][w] for w in tokens if w in features['B'])
label = argmax(p_B, p_I, p_O)
```

By construction this model is bag-of-words at the token level: it has no notion of word order, part-of-speech, or surrounding context. The project's design document names these as the three core limitations the neural models are meant to overcome.

Two real bugs fixed in the refactor

(1) A 'self.ffeatures' typo silently dropped all I-class word counts. (2) The 'I-DISEASE' literal was missing its leading pipe, so every I-class word was misrouted into the O class. Both bugs deflated the baseline; both are corrected.

5.2 BiLSTM

The neural tagger is a word-embedding Bidirectional LSTM with a time-distributed softmax classifier, built in `Train.py::build_bilstm`. The bidirectional recurrence lets each token's representation depend on both left and right context, addressing the no-sequence-information limitation of Naive Bayes.

Layer	Configuration
Input	integer token IDs, length max_len = 114
Embedding	output_dim = 50, mask_zero = True (PAD = 0)
SpatialDropout1D	rate = 0.1
Bidirectional LSTM	100 units, return_sequences = True, recurrent_dropout = 0.1
TimeDistributed Dense	n_tags units, softmax activation
Optimiser / loss	Adam (lr = 1e-3), sparse categorical cross-entropy
Training	20 epochs, batch_size = 32, validation_split = 0.1

Vocabulary is capped at the 5,000 most common training words plus PAD and UNK; out-of-vocabulary words map to UNK. The (word->index) and (tag->index) maps are pickled to `data/word2idx.pkl` and `data/tag2idx.pkl` so that the explainer and any inference step reuse exactly the same encoding.

5.3 BiLSTM-CRF

Despite the repository name, the original active model was a plain softmax BiLSTM. The refactor adds a genuine linear-chain Conditional Random Field as the output layer (`crf_layer.py`), implemented from scratch so it depends on neither `keras-contrib` nor `tensorflow-addons` (both unmaintained as of 2025).

The CRF replaces the per-token softmax with a structured output that learns label-to-label transition scores. This directly models constraints such as "I-DISEASE may only follow B-DISEASE or I-DISEASE", giving the contextual label coherence that a token-independent softmax lacks.

- Emission scores come from the TimeDistributed Dense layer (shape batch x len x tags); the CRF adds a learned transitions matrix of shape tags x tags.
- Training loss is the negative log-likelihood $-\log P(y | x)$, where the partition function $\log Z$ is computed with the forward algorithm (implemented with `tf.scan` and a log-sum-exp recurrence).
- Inference uses Viterbi decoding (a max-plus recurrence with back-pointers, back-tracked through a `tf.while_loop`) to return the single best label sequence.
- Padding is respected through the propagated Keras mask (`mask_zero = True` on the embedding), so padded positions do not contribute to loss or accuracy.

5.4 BioBERT / PubMedBERT

`train_biobert.py` fine-tunes a HuggingFace token-classification transformer - by default `microsoft/BiomedNLP-PubMedBERT-base-uncased-abstract-fulltext`, with `dmis-lab/biobert-v1.1` as an alternative. This supplies the modern baseline that the original Naive-Bayes-vs-BiLSTM comparison lacked.

- Word-level IOB labels are aligned to sub-word tokens; only the first sub-token of each word carries the label and continuation sub-tokens convert B- to I- so that seqeval decoding stays consistent.
- Defaults: 4 epochs, batch size 16, learning rate $3e-5$, max length 192; metrics are computed on the held-out test split with the same seqeval evaluator.
- With `--save-attention`, last-layer mean attention over the test set is cached to `test_attention.npy` for the interpretability comparison.

5.5 Encoding and preprocessing details

`DataPreparation.Preprocess` builds a (Sentence, Word, POS, Tag) DataFrame, filling POS via `nltk.pos_tag` token-by-token and incrementing the sentence ID on each `'.'` token; it writes `data/dfnew.csv`, read by both EDA and training. The unified `data_loader.load_job` is the refactor's replacement: it cleans the tag quirk, supports sentence-level or document-level segmentation, and returns a typed `NERDataset` dataclass shared by every modern script.

6 Evaluation Methodology

Evaluation.py now computes strict entity-level metrics with segeval under the IOB2 scheme. An entity prediction counts as correct only if both its type and its exact span boundaries match the gold annotation - the standard for NER. A token-level report (which includes the dominant 'O' class) is printed alongside, but only as a sanity check.

The metric bug that mattered most

The earlier version reported Keras 'accuracy' over a train-internal validation slice and never touched the official test split. Because ~92% of tokens are 'O', that token-level number sits near 0.98 even for a model that predicts no entities at all. Under strict entity-level segeval, an all-'O' predictor on NCBI Disease correctly scores $F1 = 0.0000$ while still showing ~0.92 token accuracy - the two numbers the old metric was conflating.

Legacy functions (PrecisionRecall, PrecisionRecallEntityLevel, PrecisionRecallEntityLevelGene) are retained as deprecated shims that forward to the new evaluator after stripping the tag quirk, so the Naive Bayes driver keeps working unchanged. multi_seed_runner.py then trains across seeds {41, 42, 43} and reports mean +/- standard deviation on precision, recall and F1 - the level of rigour reviewers expect, replacing single-run point estimates.

7 Interpretability

Interpretability is the project's distinguishing contribution. Four attribution signals are produced over the same sentences and compared directly.

7.1 LIME on the BiLSTM

Explainer.py wraps the trained BiLSTM in an eli5 TextExplainer. A MaskingTextSampler perturbs the input by replacing tokens with UNK; LIME fits a local linear surrogate to the model's behaviour around each sentence and reports per-word importance weights for each target tag (B, I, O). The explainer is position-dependent, so it explains the prediction at a specific token index.

7.2 GALE - global aggregation

Local LIME explanations are aggregated across the corpus to approximate a global view (GALE) of which words consistently push the model toward each label - moving from "why this token here" to "what the model has learned in general".

7.3 BioBERT embeddings and t-SNE

BioBertEmbeddings.py loads a BioBERT pipeline (via the nlu / spark-nlp stack), embeds sampled B / I / O tokens, and projects them to two dimensions with t-SNE, colouring points by their named-entity label. Well-separated colour clusters indicate that contextual biomedical embeddings already encode the entity distinction geometrically, before any task-specific layer is trained.

7.4 Cross-method comparison

interpretability_compare.py extracts three importance signals for a fixed set of test sentences - Naive Bayes class-conditional log-probabilities, LIME weights on the BiLSTM, and last-layer attention from PubMedBERT - and measures how much they agree, using Spearman rank correlation and top-k overlap. This reframes the work from a plain benchmark into an interpretability study.

7.5 Integrated Gradients

explainers/integrated_gradients.py adds a third, gradient-based saliency method via Captum's LayerIntegratedGradients against the transformer's input embeddings, attributing the predicted-class logit back to each token and mapping sub-word attributions back to whole words. LIME, attention, and IG together form the three methods compared in the disagreement study.

8 The Saliency-Disagreement Study

The repository scaffolds a focused research experiment that extends prior disagreement work (Krishna et al. 2022; Jukic, Tutek & Snajder, ACL 2023 Findings) into a domain those studies did not cover.

Study question

Do saliency methods disagree more on biomedical NER than on general-domain NER, and if so, which linguistic phenomena drive the gap?

8.1 Supporting modules

Module	What it provides
<code>explainers/saliency_metrics.py</code>	feature/rank/sign/signed-rank agreement @k and Spearman correlation; <code>pairwise_table()</code> emits long-form rows for aggregation
<code>explainers/phenomena.py</code>	per-sentence features: OOV rate vs. CoNLL-2003, multi-word entities, abbreviation density, parenthesised acronyms; plus <code>bucketize()</code>
<code>explainers/faithfulness.py</code>	deletion / insertion AUC against any <code>score_fn</code> - a sanity check that a disagreement gap is not just one method failing
<code>run_disagreement_study.py</code>	driver across NCBI, BC5CDR and CoNLL-2003 x {LIME, IG, attention}; writes long-form and summary CSVs

8.2 Reference numbers (smoke test)

On the NCBI Disease test set (981 sentences), the phenomena module reports multi-word entities in 31.5% of sentences, mean abbreviation density 0.17, and '(ABBR)' acronym patterns in 11.1% of sentences. The CoNLL-2003 equivalents are typically below 5% for multi-word entities and near 0% for parenthesised abbreviations - so the cross-domain gap exists in the data before any model is trained, which is exactly the premise the study sets out to test on explanations.

9 The 2026 Refactor: What Changed and Why

Three issues in the original code would have been flagged in peer review; the refactor addresses each:

1. Reported F1 (~0.98) was Keras accuracy over a train-internal validation slice; the held-out NCBI test split was never evaluated. Now fixed with `segeval` on the real test set.
2. The "BiLSTM-CRF" named in the title was a softmax BiLSTM in the code. A real CRF layer is now wired in via `crf_layer.py` and `Train.build_bilstm_crf`.
3. The only baselines were 1990s-era Naive Bayes vs. a 2016-era BiLSTM. A modern BioBERT / PubMedBERT baseline is now provided via `train_biobert.py`.

Additional improvements

- `data_loader.py`: one reader for NCBI Disease, BC5CDR and CoNLL-2003, with sentence- or document-level segmentation and IOB1->IOB2 conversion.
- `Evaluation.py`: `segeval`-based, with deprecated shims so `main_NB.py` still runs.
- `multi_seed_runner.py`: mean +/- std across seeds on every metric.
- `interpretability_compare.py` and the `explainers/` package: the analytical core.
- `NaiveBayes.py`: the two correctness bugs described in Section 5.1 fixed.

Honest reporting of what was NOT run

The refactor was prepared without GPU, without HuggingFace Hub access, and with no pre-installed TensorFlow/PyTorch. The data loader and `segeval` evaluator are smoke-tested end-to-end (sentence counts match the published split within tokenisation noise; the all-'O' predictor scores entity-F1 = 0). None of the BiLSTM / BiLSTM-CRF / BioBERT / multi-seed numbers were regenerated, so the results table deliberately leaves those rows as TBD rather than carrying forward the inflated token-level values.

10 Results

10.1 Re-run targets

The results table is intentionally a set of targets to be filled by a GPU re-run. Entity-level (strict IOB2) precision, recall and F1 on the NCBI Disease test set are the reporting standard.

Model	Precision	Recall	F1
Naive Bayes (per-token, no context)	TBD	TBD	TBD
BiLSTM (this repo, retrained)	TBD	TBD	TBD
BiLSTM-CRF (this repo, retrained)	TBD	TBD	TBD
BioBERT v1.1 (HuggingFace)	TBD	TBD	TBD
PubMedBERT abstract+fulltext	TBD	TBD	TBD

10.2 Historical training curves (legacy BiLSTM)

The figures below are the accuracy and loss curves saved by an original 20-epoch BiLSTM run. They illustrate the metric trap of Section 6: training accuracy climbs toward 1.0 and validation accuracy plateaus near 0.99 - numbers that look excellent only because they are dominated by the majority 'O' class. The loss curves also show the validation loss flattening and slightly rising after a few epochs, the usual sign of mild over-fitting.

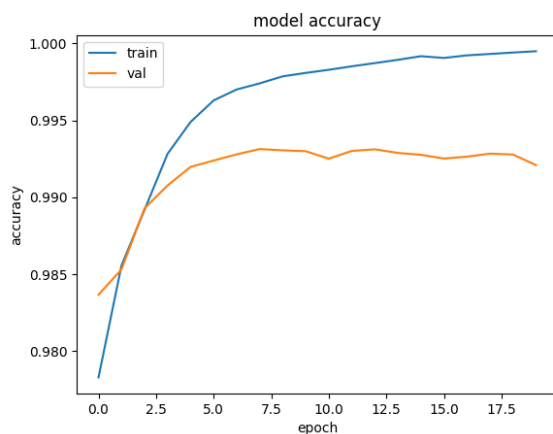


Figure 1a. Training vs. validation accuracy.

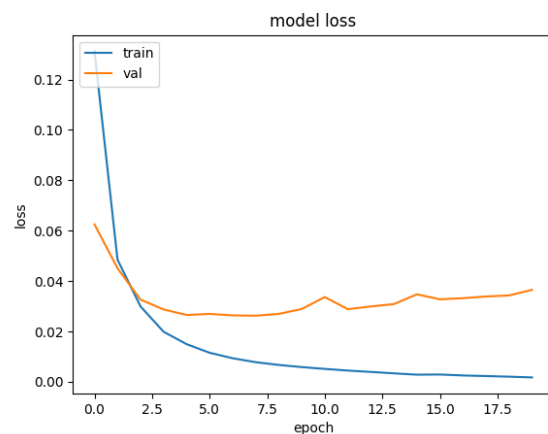


Figure 1b. Training vs. validation loss.

These plots are retained as an artifact and a cautionary example; the authoritative comparison is the entity-level table once regenerated.

11 How to Run and Reproduce

All commands assume the working directory is code/.

Naive Bayes baseline (CPU)

```
cd code
python main_NB.py
```

BiLSTM / BiLSTM-CRF

```
python main_NN.py --model bilstm --epochs 20 --seed 42
python main_NN.py --model bilstm_crf --epochs 20 --seed 42

# multi-seed mean +/- std
python multi_seed_runner.py --model bilstm_crf --seeds 41 42 43 --epochs 20
```

BioBERT / PubMedBERT (GPU + HuggingFace)

```
python train_biobert.py \
  --train ../data/ner-disease/train.iob \
  --dev ../data/ner-disease/dev.iob \
  --test ../data/ner-disease/test.iob \
  --model microsoft/BiomedNLP-PubMedBERT-base-uncased-abstract-fulltext \
  --epochs 4 --batch-size 16 --seed 42 --save-attention
```

Interpretability comparison and disagreement study

```
python interpretability_compare.py \
  --bilstm-weights '../models/bilstm_seed42_*.h5' \
  --biobert-dir ../models/biobert_ncbi --n-sentences 100

python run_disagreement_study.py \
  --datasets ncbi bc5cdr conll2003 \
  --bilstm-weights '../models/bilstm_crf_seed42_*.h5' \
  --biobert-dir ../models/biobert_ncbi --bert-dir ../models/bert_conll2003 \
  --n-sentences 200 --out-dir ../results
```

Operational notes

Run every script from inside code/ (paths are relative). The original scripts have side effects at import time and the explainer hard-codes a checkpoint filename (update it after retraining). Train.py also calls wandb.init; log in, run 'wandb offline', or remove the call before training. BC5CDR and CoNLL-2003 must be placed locally as they are not redistributed.

12 Environment and Dependencies

Developed and tested with Python 3.9.6 on Linux and Windows; install with `pip install -r requirements.txt`. The stack splits along framework lines so the TF and PyTorch dependencies stay independent.

Area	Key packages
Neural (Keras path)	tensorflow 2.9, keras 2.9
Transformers	torch, transformers, datasets (BioBERT/PubMedBERT)
Evaluation	segeval (strict IOB2), scikit-learn
Interpretability	lime, eli5, captum, nlu / spark-nlp (BioBERT t-SNE)
Data / NLP	pandas, numpy, nltk (POS tagging)
Viz / tracking	matplotlib, seaborn, wandb

Experiment tracking uses Weights & Biases (project GALE_LIME_NER_LSTM_CRF_DISEASE, entity robofied). The BioBERT t-SNE step additionally requires a configured spark-nlp / nlu environment with Java.

13 Limitations and Future Work

13.1 Known limitations

- Headline benchmark numbers are not yet regenerated under the corrected metric; the results table is TBD pending a GPU run.
- The BiLSTM vocabulary is capped at 5,000 words, so rare biomedical terms collapse to UNK - a real constraint given the high OOV rate of the domain.
- `max_len = 114` is hard-coded; longer abstracts are truncated in sentence mode.
- Original scripts execute at import time and the LIME explainer hard-codes a checkpoint name, so retraining requires manual edits.
- The legacy and modern tag conventions coexist (literal '|TAG\n' vs. canonical form), a foot-gun for anyone editing the Naive Bayes path.

13.2 Future work

- Regenerate the full entity-level results table across all four model families with multi-seed mean +/- std.
- Complete the disagreement study: run all three saliency methods across the three datasets and add paired-bootstrap significance tests.
- Replace the capped-vocabulary embedding with sub-word or contextual embeddings to reduce the UNK problem in the BiLSTM path.
- Add a proper `__main__` guard and configuration file to remove import-time side effects and hard-coded checkpoint names.

14 Conclusion

This project is best understood not as a single model but as a comparative study of disease-entity recognition across four eras of NER methodology, unified by a shared evaluation protocol and, crucially, by an interpretability layer that asks what each model has actually learned. The from-scratch Naive Bayes baseline establishes a context-free floor; the BiLSTM adds sequential memory; the CRF adds structured label coherence; and BioBERT/PubMedBERT brings domain-pretrained contextual representations.

The 2026 refactor turned an over-optimistic demonstration into a defensible experimental scaffold: strict entity-level metrics, a real CRF, a modern transformer baseline, multi-seed reporting, and a cross-method saliency-disagreement framework. With the benchmark re-run completed, the code base is positioned to answer its central question - what sequence and transformer models see in biomedical NER that count-based methods cannot - with evidence rather than with an inflated accuracy score.

Prepared as project documentation for "Interpreting Bidirectional LSTM-CRF for Disease Entity Recognition" by Akshat Gupta and Silvia Cunico, University of Stuttgart. Statistics and architecture details in this report were extracted directly from the repository sources.